

GUIA DE ESTUDO

FRONTEND DO SISTEMA DE CONTROLE DE ESTOQUE

HTML • CSS • JavaScript • DOM • Fetch API

Resumo do projeto, roteiro de apresentação e 60 perguntas com respostas

Equipe 02 — Arquitetura de Sistemas Ciberfísicos e Modelagem

1. O que é o frontend deste projeto?

A parte visual que recebe ações do usuário e conversa com o backend.

O frontend é executado no navegador. Ele mostra as telas, captura cliques e dados digitados, chama a API Flask e transforma as respostas JSON em tabelas, cartões e mensagens. Neste projeto ele foi construído com HTML, CSS e JavaScript puro, sem framework.

Atenção ao nome da linguagem

O código do navegador é JavaScript, e não Java. Java e JavaScript são linguagens diferentes. O projeto não usa Java no frontend.

Responsabilidade de cada tecnologia

- HTML: cria a estrutura das páginas — menus, títulos, tabelas, campos, botões e modais.
- CSS: define cores, tamanhos, espaçamentos, grid, estados visuais e aparência dos modais.
- JavaScript: aplica o comportamento — eventos, validações, requisições, busca e atualização da tela.
- Fetch API: faz as requisições HTTP ao backend Flask.
- DOM: é a representação da página que o JavaScript consulta e modifica.

Visão do fluxo

```
Usuário clica ou preenche um campo
↓
JavaScript captura o evento e valida o básico
↓
api.js envia HTTP para http://127.0.0.1:5000
↓
Flask consulta/atualiza SQLite e devolve JSON
↓
JavaScript atualiza o DOM e mostra o resultado
```

2. Organização das telas e arquivos

O projeto separa estrutura, aparência, comunicação e regra visual.

Arquivo	Responsabilidade
index.html	Dashboard: totais, alertas e uma tabela resumida de produtos.
produtos.html	Cadastro, busca, exclusão, entrada, saída e histórico de produto.
categorias.html	Cadastro, listagem e pesquisa de categorias.
relatorios.html	Relatórios de inventário e movimentações.
css/style.css	Estilo compartilhado de todas as páginas.
js/api.js	Funções que fazem todas as chamadas HTTP para o Flask.
js/app.js	Comportamento específico do dashboard.
js/produtos.js	Tabela, cadastro, busca, exclusão e histórico.
js/estoque.js	Modal e envio de entradas e saídas.
js/categorias.js	Tela de categorias.
js/relatorios.js	Montagem dos dois relatórios.

Separação simples

O arquivo api.js concentra o acesso à API. Os outros scripts cuidam da tela em que são carregados. Essa separação reduz repetição e é adequada ao nível acadêmico do trabalho.

3. HTML, DOM e eventos

Como a estrutura ganha comportamento no navegador.

Os elementos importantes possuem id ou classe. O JavaScript usa métodos como `document.getElementById` e `querySelectorAll` para encontrá-los. Depois, usa `addEventListener` para reagir a cliques e digitação.

```
document.addEventListener("DOMContentLoaded", () => {  
  carregarProdutos();  
});  
  
btnSalvar.addEventListener("click", salvarProduto);  
campoPesquisa.addEventListener("input", pesquisarProdutos);
```

Por que esperar DOMContentLoaded?

Esse evento indica que o HTML já foi interpretado. Assim, o script pode procurar botões, tabelas e campos sem receber null porque o elemento ainda não existe.

Como a tabela é atualizada?

O JavaScript limpa o corpo da tabela e cria linhas com `innerHTML`. Depois da renderização, adiciona eventos aos botões recém-criados. Quando uma operação termina, a listagem é carregada novamente para refletir o banco.

Ponto técnico para a banca

Os scripts são incluídos em ordem. `api.js` vem antes dos scripts das telas, por isso funções como `listarProdutos()` já existem quando são chamadas. Em `Produtos`, `estoque.js` é carregado depois, mas sua função só é usada quando o usuário clica — nesse momento o arquivo já terminou de carregar.

HTML bem iniciado, mas ainda melhorável

- O projeto usa `lang='pt-BR'`, UTF-8 e a meta viewport.
- Os modais são estruturas escondidas que recebem ou removem a classe `hidden`.
- Alguns itens do menu são elementos `li` com `onclick`; links ou botões seriam mais semânticos e acessíveis.
- Há botões apenas com ícones; textos visíveis ou `aria-label` melhorariam a acessibilidade.

4. CSS e identidade visual

Uma folha de estilos mantém as páginas com a mesma aparência.

O layout principal usa Flexbox: uma barra lateral fixa de 250 px e uma área de conteúdo flexível. Os cartões do dashboard usam CSS Grid com quatro colunas. Tabelas, botões e modais reutilizam classes.

Uso	Cor	Código
Barra lateral		#1E293B
Ação principal		#2563EB
Entrada		#22C55E
Saída		#F59E0B
Exclusão/alerta		#EF4444
Fundo		#F4F7FE

Classes importantes

- `.layout`, `.sidebar` e `.content`: estrutura geral.
- `.cards` e `.card`: indicadores do dashboard.
- `.table-container`: bloco branco que envolve tabelas.
- `.modal` e `.modal-content`: sobreposição e janela de formulário.
- `.hidden`: usa opacidade, visibilidade e pointer-events para esconder.
- `.btn-entrada`, `.btn-saida` e `.btn-excluir`: cores que reforçam a ação.

Limitação real

Embora exista meta viewport, o CSS não possui `@media` queries. Portanto, o layout ainda não é totalmente responsivo em telas pequenas. Uma melhoria seria adaptar sidebar, grid e tabelas para celular.

5. JavaScript, Fetch API e JSON

A camada `api.js` é a ponte entre navegador e Flask.

As funções são `async` porque uma requisição leva tempo. O `await` espera a resposta sem travar toda a página. O `fetch` envia a URL, o método, os cabeçalhos e, quando necessário, o corpo em JSON.

```
async function cadastrarProduto(produto) {
  const resposta = await fetch(`${API_URL}/produtos`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(produto)
  });
  const dados = await resposta.json();
  if (!resposta.ok) throw new Error(dados.erro);
  return dados;
}
```

Conceitos que aparecem nesse trecho

- HTTP: protocolo usado para frontend e backend se comunicarem.
- GET: consulta dados; POST: cadastra ou registra movimentação; DELETE: desativa produto.
- Content-Type application/json: informa o formato enviado.
- JSON.stringify: transforma um objeto JavaScript em texto JSON.
- resposta.json(): converte o JSON recebido em objeto JavaScript.
- resposta.ok: indica status HTTP entre 200 e 299.
- throw new Error: encaminha o erro para o catch da tela.

Por que há validação nos dois lados?

A validação do frontend melhora a experiência e evita pedidos claramente inválidos. Porém ela pode ser contornada. O backend precisa validar novamente porque é a parte confiável do sistema.

6. Fluxos principais do sistema

O que acontece do clique até a atualização visual.

Dashboard

Ao abrir, consulta produtos, estoque baixo e indicadores. Preenche quatro cartões e uma tabela com cinco produtos.

Cadastro de produto

Abre modal, carrega categorias, lê campos, valida, envia POST /produtos, fecha o modal e recarrega a tabela.

Entrada de estoque

Seleciona produto e tipo, pede quantidade e observação, envia POST /produtos/{id}/entrada e atualiza a lista.

Saída de estoque

Usa fluxo parecido, mas chama /saida. Se faltar saldo, o backend devolve erro e a tela mostra a mensagem.

Exclusão

Pede confirmação e envia DELETE. O produto é inativado no backend, preservando seu histórico.

Histórico

Consulta /produtos/{id}/historico e monta a tabela do modal.

Categorias

Lista, filtra no navegador e envia uma nova categoria para POST /categorias.

Relatórios

Consulta inventário e movimentações. Um mapa de produtos ajuda a mostrar nomes no lugar dos IDs.

7. Como demonstrar, testar e explicar limites

Um roteiro seguro para apresentar o frontend.

Roteiro de demonstração

- Inicie o Flask e abra `http://127.0.0.1:5000`.
- Mostre o dashboard e explique que os números vêm da API, não estão escritos no HTML.
- Cadastre categoria e produto; faça entrada e saída; confira saldo e histórico.
- Tente retirar mais do que existe para mostrar a validação do backend aparecendo na interface.
- Abra os relatórios e mostre que os dados permanecem após atualizar a página.

Ferramentas do navegador

- Console: mostra erros JavaScript.
- Network/Rede: mostra URL, método, status, corpo enviado e JSON recebido.
- Elements/Elementos: permite inspecionar HTML e estilos aplicados.
- Disable cache: ajuda a garantir que o navegador carregou a versão nova durante testes.

Limitações que podem ser assumidas com honestidade

- Não há login; o texto “Administrador” é apenas visual.
- Não há paginação; as tabelas carregam todos os registros.
- O endereço da API está fixo em `127.0.0.1:5000`.
- Uso de `innerHTML` com dados externos pede sanitização para evitar XSS em um sistema real.
- Faltam melhorias de acessibilidade: navegação por teclado, foco no modal e atributos ARIA.
- A busca é local e simples; em listas grandes seria melhor pesquisar no backend.

Pegadinha do dashboard

“Produtos Recentes” não usa data de cadastro. O backend devolve produtos em ordem alfabética e o frontend aplica `slice(0, 5)`. O nome correto hoje seria “Resumo de Produtos” ou seria necessário armazenar e ordenar pela data.

8. Perguntas — HTML e CSS

Fundamentos visuais e estrutura das páginas.

1. O que é frontend?

Resposta: É a parte do sistema executada no navegador e usada diretamente pelo usuário.

2. Quais tecnologias foram usadas?

Resposta: HTML para estrutura, CSS para aparência e JavaScript para comportamento e integração.

3. O projeto usa Java?

Resposta: Não. Ele usa JavaScript, que é uma linguagem diferente de Java.

4. Para que serve o HTML?

Resposta: Para organizar elementos como menu, títulos, tabelas, campos, botões e modais.

5. Para que serve o CSS?

Resposta: Para controlar cores, tamanhos, espaçamentos, layout e estados visuais.

6. O que é Flexbox?

Resposta: Um modelo de layout usado aqui para posicionar sidebar e conteúdo.

7. O que é CSS Grid?

Resposta: Um modelo de grade usado nos cartões do dashboard.

8. O que é uma classe CSS?

Resposta: Um nome reutilizável que aplica o mesmo conjunto de estilos a vários elementos.

9. O que é um id no HTML?

Resposta: Um identificador único, usado pelo JavaScript para localizar um elemento específico.

10. O layout é responsivo?

Resposta: Parcialmente. Há viewport, mas faltam media queries para adaptação completa a telas pequenas.

11. De onde vêm os ícones?

Resposta: Da biblioteca Font Awesome carregada por CDN.

12. De onde vem a fonte Inter?

Resposta: Do Google Fonts; se a internet falhar, o navegador usa uma fonte alternativa sans-serif.

9. Perguntas — JavaScript e DOM

Eventos, modais, busca e renderização.

13. O que é JavaScript neste projeto?

Resposta: É a linguagem que captura eventos, valida campos, chama a API e atualiza a página.

14. O que é DOM?

Resposta: É a representação em objetos do HTML que o JavaScript pode ler e modificar.

15. Por que usar `DOMContentLoaded`?

Resposta: Para executar o código depois que os elementos HTML já estiverem disponíveis.

16. O que faz `addEventListener`?

Resposta: Associa uma função a um evento, como click ou input.

17. Qual a diferença entre click e input?

Resposta: Click ocorre ao acionar um elemento; input ocorre a cada mudança no campo.

18. Como a busca funciona?

Resposta: Compara o texto digitado com células das linhas e oculta as que não correspondem.

19. O que faz `innerHTML`?

Resposta: Lê ou substitui o conteúdo HTML interno de um elemento.

20. Por que os eventos dos botões da tabela são adicionados após renderizar?

Resposta: Porque esses botões são criados dinamicamente e não existiam antes.

21. Como um modal é aberto?

Resposta: O JavaScript remove a classe `hidden` e preenche os dados necessários.

22. Como um modal é fechado?

Resposta: A classe `hidden` é adicionada novamente e o formulário pode ser limpo.

23. Por que usar `Number` no valor digitado?

Resposta: Porque valores de input chegam como texto e a quantidade precisa ser numérica.

24. O que é `try/catch`?

Resposta: É uma estrutura que tenta executar uma operação e trata o erro caso ela falhe.

10. Perguntas — API e assincronismo

Fetch, HTTP, JSON e tratamento de erros.

25. O que é uma API?

Resposta: É um conjunto de rotas e regras que permite a comunicação entre frontend e backend.

26. O que é fetch?

Resposta: É uma função do navegador que envia requisições HTTP.

27. Por que as funções são async?

Resposta: Porque chamadas de rede são assíncronas e não retornam imediatamente.

28. O que faz await?

Resposta: Espera a Promise terminar dentro de uma função async, deixando o código mais legível.

29. O que é JSON?

Resposta: Um formato textual leve usado para trocar dados entre navegador e Flask.

30. O que faz JSON.stringify?

Resposta: Converte um objeto JavaScript em texto JSON para envio.

31. O que faz resposta.json()?

Resposta: Converte o corpo JSON recebido em um objeto JavaScript.

32. Quais métodos HTTP são usados?

Resposta: GET para consultar, POST para criar/movimentar e DELETE para excluir logicamente.

33. O que significa resposta.ok?

Resposta: Que o status HTTP está entre 200 e 299.

34. Como o erro do backend chega à tela?

Resposta: api.js lança Error com a mensagem recebida e o catch mostra essa mensagem.

35. Por que api.js vem antes dos outros scripts?

Resposta: Para suas funções de comunicação já estarem declaradas quando as telas precisarem delas.

36. Por que o endereço da API está em uma constante?

Resposta: Para centralizar a URL; em produção seria melhor configurá-la por ambiente ou usar rotas relativas.

11. Perguntas — Funcionamento do projeto

Respostas ligadas diretamente às telas implementadas.

37. Como o dashboard obtém seus números?

Resposta: Ele chama /dashboard e escreve os valores retornados nos cartões.

38. Como é mostrado o estoque baixo?

Resposta: A API retorna os produtos com quantidade menor ou igual ao mínimo e a tela mostra o total.

39. Como o cadastro carrega categorias?

Resposta: A tela consulta /categorias e cria opções no select.

40. O saldo é calculado no frontend?

Resposta: Não. O backend altera e valida o saldo; o frontend apenas mostra o resultado.

41. O que acontece depois de uma entrada?

Resposta: A mensagem é mostrada, o modal fecha e a lista de produtos é recarregada.

42. Quem impede uma saída maior que o saldo?

Resposta: O backend. O frontend envia o pedido e apresenta o erro devolvido.

43. Como o histórico é exibido?

Resposta: Uma rota retorna movimentações e o JavaScript cria as linhas no modal.

44. A exclusão apaga o histórico?

Resposta: Não. O frontend solicita DELETE e o backend inativa o produto, preservando registros.

45. Como o relatório troca ID por nome?

Resposta: Cria um mapa de produtos por ID e consulta esse mapa ao montar as linhas.

46. Por que atualizar a tabela após salvar?

Resposta: Para buscar novamente o estado real armazenado no banco.

47. Por que há mensagens de lista vazia?

Resposta: Para informar ao usuário que a requisição funcionou, mas não há registros.

48. Os produtos do dashboard são realmente os mais recentes?

Resposta: Não. Atualmente são os cinco primeiros da lista ordenada por nome.

12. Perguntas difíceis e melhorias

Limites técnicos explicados com honestidade.

49. Por que validar também no backend?

Resposta: Porque o frontend pode ser alterado ou ignorado pelo usuário; só ele não protege os dados.

50. Existe autenticação?

Resposta: Não. “Administrador” é apenas um texto da interface.

51. O que é XSS?

Resposta: É a execução indevida de script inserido como dado. Usar innerHTML sem sanitização pode criar esse risco.

52. Como reduzir o risco de XSS?

Resposta: Usar textContent para texto, criar elementos pelo DOM e sanitizar conteúdo quando HTML for necessário.

53. O que melhoraria a acessibilidade?

Resposta: Links e botões semânticos, aria-label, foco no modal, tecla Escape e navegação por teclado.

54. Como melhorar a responsividade?

Resposta: Adicionar media queries, recolher a sidebar e permitir rolagem adequada nas tabelas.

55. Como descobrir um erro de integração?

Resposta: Abrir Network e Console, conferir URL, método, status, JSON enviado e resposta.

56. O que significa erro 404?

Resposta: Que a rota ou recurso solicitado não foi encontrado.

57. O que significa erro 500?

Resposta: Que ocorreu uma falha interna no servidor; é preciso olhar o terminal do Flask.

58. O que acontece se o Flask estiver desligado?

Resposta: O fetch falha e o catch apresenta erro; não há acesso aos dados.

59. O sistema escala para milhares de produtos?

Resposta: Ainda não bem, pois não há paginação e muita renderização é feita com innerHTML em laços.

60. Qual seria a próxima melhoria mais importante?

Resposta: Responsividade, acessibilidade, sanitização, configuração da URL e paginação, mantendo o backend como fonte da verdade.

13. Resumo de bolso para a apresentação

Leia pouco antes de apresentar.

- ☐ O frontend foi feito com HTML, CSS e JavaScript puro.
- ☐ O navegador chama a API Flask com fetch e recebe JSON.
- ☐ O arquivo api.js centraliza as requisições; cada outro script cuida de uma tela.
- ☐ O frontend não calcula o saldo definitivamente: o backend valida e persiste.
- ☐ Depois de salvar, a tela consulta novamente a API para mostrar o estado real.
- ☐ O dashboard mostra totais, entradas, saídas, estoque baixo e um resumo de produtos.
- ☐ As buscas atuais acontecem no navegador e são adequadas ao volume acadêmico.
- ☐ Os modais usam a classe hidden para abrir e fechar.
- ☐ Erros da API são lançados em api.js e exibidos pelo catch das telas.
- ☐ O sistema ainda pode melhorar em responsividade, acessibilidade, segurança contra XSS e paginação.

Frase de abertura sugerida

“Nosso frontend foi desenvolvido com tecnologias básicas da web. O HTML organiza as telas, o CSS mantém a identidade visual e o JavaScript integra a interface à API Flask. As operações são persistidas no SQLite pelo backend, e a tela sempre apresenta os dados recebidos da API.”

Se você não souber uma resposta

Explique o que está implementado e diga com tranquilidade que o ponto pode ser uma evolução. Não invente funcionalidades. Demonstrar que você conhece os limites do projeto costuma ser melhor do que afirmar algo incorreto.